

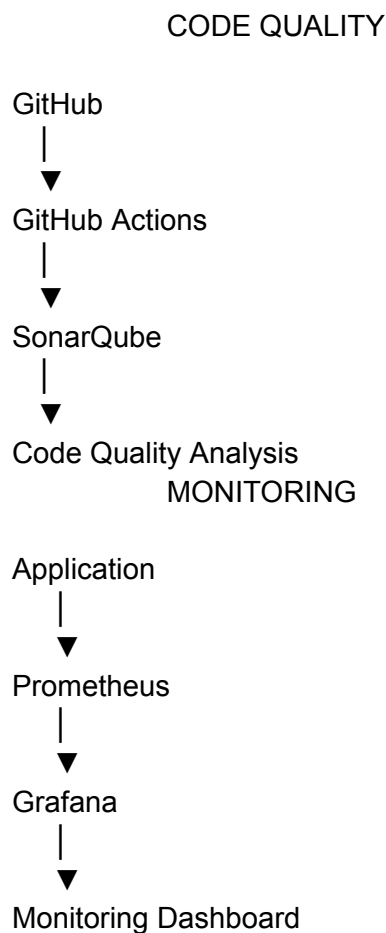
DAY 5 – SONARQUBE, PROMETHEUS & GRAFANA

1. Day 5 Objective

By the end of Day 5, students will understand:

- Code quality using **SonarQube**
- Metrics collection using **Prometheus**
- Monitoring and visualization using **Grafana**
- How these tools fit into a DevOps workflow
- How Docker can be used to run DevOps tools locally

Day 5 Architecture



For this workshop, **SonarQube, Prometheus and Grafana are running locally on Windows using Docker Desktop.**

EC2 is not required for these tools.

PART 1 – SONARQUBE

2. What is SonarQube?

SonarQube is a code quality and code security analysis platform.

It analyzes source code and helps identify:

- Bugs
- Vulnerabilities
- Security issues
- Code smells
- Maintainability problems
- Code duplication
- Test coverage

Simple explanation

Source Code



SonarQube Analysis



Code Quality Report

3. Start SonarQube Using Docker

WINDOWS POWERSHELL

Create a folder:

```
mkdir C:\devops-day5  
cd C:\devops-day5
```

Start SonarQube:

```
docker run -d --name sonarqube -p 9000:9000 sonarqube:community
```

Check the container:

`docker ps`

Expected:

`sonarqube`

with:

`0.0.0.0:9000->9000/tcp`

4. Open SonarQube

Open the browser:

`http://localhost:9000`

Default login:

Username: admin

Password: admin

SonarQube may ask you to change the password after the first login.

5. Create a SonarQube Project

In SonarQube:

Projects



Create Project



Create project manually

Use:

Project Name: DevOps React Demo

Project Key: devops-react-demo

For the **New Code Definition**, use:

Follows the instance's default

Then click:

Create project

6. Generate SonarQube Token

Go to the local analysis setup.

Select:

Locally

Generate a project token.

Example token name:

Analyze "devops-react-demo"

Expiration:

30 days

Click:

Generate

Important

The token is a secret.

Do not share the token publicly.

PART 2 – CONNECT REACT PROJECT TO SONARQUBE

7. React Project

Our React project is located at:

D:\Day3KIs\react-deployment

WINDOWS POWERSHELL

Go to the project:

```
cd D:\Day3KIs\react-deployment
```

Check the project:

```
Get-ChildItem
```

Expected files/folders include:

```
src
public
package.json
package-lock.json
vite.config.js
Dockerfile
.github
```

8. PowerShell npm Issue

On some Windows systems, running:

```
npm --version
```

may produce:

```
npm.ps1 cannot be loaded because running scripts is disabled
```

Instead use:

```
npm.cmd --version
```

Node version:

```
node --version
```

There is no need to change the Windows PowerShell execution policy for this workshop.

9. Create SonarQube Configuration

Inside:

D:\Day3KIs\react-deployment

create:

sonar-project.properties

WINDOWS POWERSHELL

notepad sonar-project.properties

Add:

```
sonar.projectKey=devops-react-demo  
sonar.projectName=DevOps React Demo  
sonar.sources=src  
sonar.exclusions=**/node_modules/**,**/dist/**
```

Save the file.

Verify:

Get-Content sonar-project.properties

Expected:

```
sonar.projectKey=devops-react-demo  
sonar.projectName=DevOps React Demo  
sonar.sources=src  
sonar.exclusions=**/node_modules/**,**/dist/**
```

10. Create Docker Network

We use a Docker network so that the containers can communicate with each other.

WINDOWS POWERSHELL

docker network create devops-day5

Connect SonarQube to the network:

docker network connect devops-day5 sonarqube

Verify:

docker network inspect devops-day5

SonarQube should appear under:

Containers

11. Run SonarScanner Using Docker

We don't need to install SonarScanner directly on Windows.

The scanner can run inside Docker.

WINDOWS POWERSHELL

From:

D:\Day3KIs\react-deployment

run:

```
docker run --rm `
  --network devops-day5 `
  -v "${PWD}:/usr/src" `
  -w /usr/src `
  -e SONAR_HOST_URL="http://sonarqube:9000" `
  -e SONAR_TOKEN="YOUR_TOKEN_HERE" `
  sonarsource/sonar-scanner-cli
```

Replace:

YOUR_TOKEN_HERE

with the token generated in SonarQube.

Important

Do not share the actual token.

12. SonarScanner Explanation

`--network devops-day5`

Connects the scanner to the Docker network.

`-v "${PWD}:/usr/src"`

Makes the React project available inside the scanner container.

`-w /usr/src`

Sets the scanner working directory.

`SONAR_HOST_URL=http://sonarqube:9000`

Tells the scanner where SonarQube is running.

`SONAR_TOKEN`

Authenticates the scanner.

`sonarsource/sonar-scanner-cli`

Runs the SonarScanner Docker image.

13. Successful SonarQube Analysis

A successful scan will contain:

`ANALYSIS SUCCESSFUL`

and:

`EXECUTION SUCCESS`

The analysis results will then be available in SonarQube.

14. View SonarQube Results

Open:

<http://localhost:9000>

Go to:

Projects



DevOps React Demo

The dashboard can show:

- Security
- Reliability
- Maintainability
- Code Smells
- Vulnerabilities
- Security Hotspots
- Coverage
- Duplications
- Quality Gate

Our React project result

The analysis produced:

Lines of Code: 338

Security: A

Reliability: A

Maintainability: A

Security Hotspots: 0

Duplications: 0.0%

Coverage: 0.0%

The 0% coverage is expected because we haven't configured automated tests for this React project.

PART 3 – PROMETHEUS

15. What is Prometheus?

Prometheus is a monitoring and metrics collection system.

It collects numerical information called **metrics** from configured targets.

Examples of metrics include:

CPU usage
Memory usage
Request count
Response time
Application status

Simple explanation

Application / Service



Metrics



Prometheus

Prometheus periodically **scrapes** metrics from targets.

16. Prometheus Configuration

We keep Prometheus configuration separate from the React project.

Create:

C:\devops-day5\prometheus.yml

Do **not** put it inside:

D:\Day3KIs\react-deployment

because the React project and monitoring infrastructure are separate concerns.

17. Create prometheus.yml

WINDOWS POWERSHELL

```
cd C:\devops-day5
```

Create the file:

```
notepad prometheus.yml
```

Add:

```
global:
```

```
  scrape_interval: 5s
```

```
scrape_configs:
```

```
  - job_name: "prometheus"
```

```
    static_configs:
```

```
      - targets: ["localhost:9090"]
```

Save it.

Verify:

```
Get-Content prometheus.yml
```

18. Start Prometheus

WINDOWS POWERSHELL

```
docker run -d --name prometheus `
```

```
  --network devops-day5 `
```

```
  -p 9090:9090 `
```

```
  -v "C:\devops-day5\prometheus.yml:/etc/prometheus/prometheus.yml" `
```

```
  prom/prometheus
```

Check containers:

```
docker ps
```

You should see:

prometheus
sonarqube

Prometheus is available at:

<http://localhost:9090>

19. Verify Prometheus

Open:

<http://localhost:9090>

Go to:

Status



Targets

The Prometheus target should show:

UP

This means Prometheus can successfully scrape the configured target.

Important concept

UP = 1

DOWN = 0

The **up** metric is useful for checking whether a scrape target is reachable.

PART 4 – GRAFANA

20. What is Grafana?

Grafana is a visualization and dashboard platform.

Grafana does not normally collect the metrics itself.

Instead:

Prometheus



Metrics



Grafana



Dashboard

Simple explanation

Prometheus collects the metrics. Grafana visualizes those metrics.

21. Start Grafana

We initially had an old Grafana container. It was stopped, so we removed it.

WINDOWS POWERSHELL

Remove old container:

```
docker rm grafana
```

Start a fresh Grafana container:

```
docker run -d --name grafana `
  --network devops-day5 `
  -p 3000:3000 `
  grafana/grafana
```

Check:

```
docker ps
```

We should now have:

```
sonarqube
prometheus
grafana
```

22. Open Grafana

Open:

`http://localhost:3000`

Default login:

Username: admin

Password: admin

Grafana may ask you to change the password.

23. Connect Grafana to Prometheus

Inside Grafana:

Connections



Data Sources



Add data source



Prometheus

For the Prometheus server URL, use:

`http://prometheus:9090`

Do NOT use:

`http://localhost:9090`

because Grafana is running inside a Docker container.

Inside the Docker network, Grafana can find Prometheus using its container name:

grafana



`prometheus:9090`

Click:

Save & test

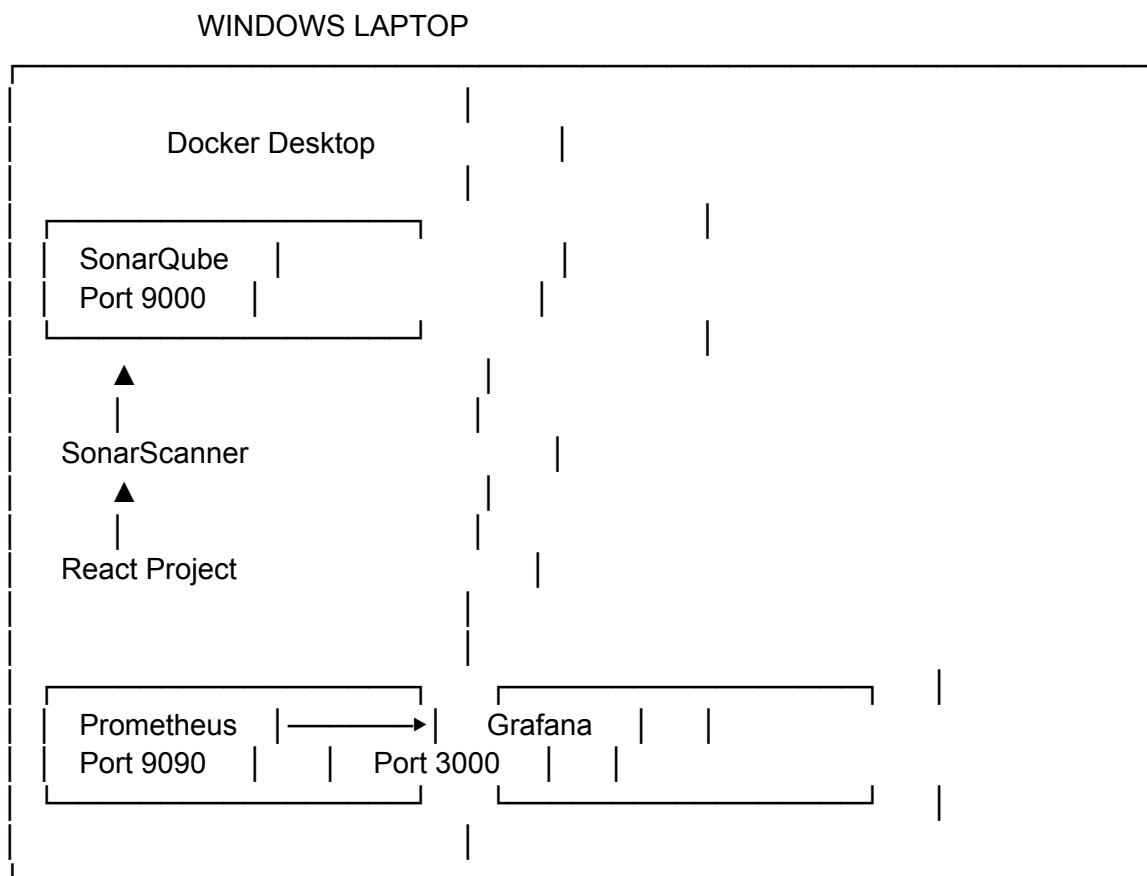
Expected result:

Successfully queried the Prometheus API.

This confirms that Grafana can communicate with Prometheus.

24. Current Day 5 Architecture

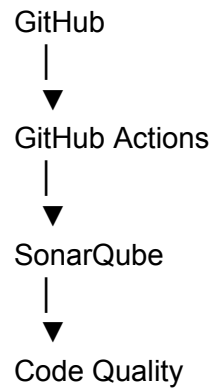
At this point, the local environment is:



25. Complete Day 5 Concept

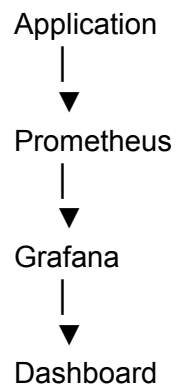
You can explain the overall workflow to students like this:

CODE QUALITY



And:

MONITORING



Key difference

Tool	Purpose
GitHub	Source code management
GitHub Actions	CI/CD automation
SonarQube	Code quality and security analysis
Prometheus	Metrics collection
Grafana	Metrics visualization
Docker	Containerization
AWS EC2	Cloud deployment

26. Commands Completed So Far

SonarQube

```
docker run -d --name sonarqube -p 9000:9000 sonarqube:community
```

Docker network

```
docker network create devops-day5  
docker network connect devops-day5 sonarqube
```

Prometheus

```
docker run -d --name prometheus `\  
  --network devops-day5 `\  
  -p 9090:9090 `\  
  -v "C:\devops-day5\prometheus.yml:/etc/prometheus/prometheus.yml" `\  
  prom/prometheus
```

Grafana

```
docker rm grafana  
docker run -d --name grafana `\  
  --network devops-day5 `\  
  -p 3000:3000 `\  
  grafana/grafana
```

Check everything

```
docker ps
```

Expected:

```
sonarqube  
prometheus  
grafana
```

Current Status